

Object-Oriented Programming Study

2021/02/02 ~

박지원

목차

1강_C++ 기초

2강_클래스의 기본

3강_상속

4강_다형성

5강_가상함수

6강_순수가상함수와 추상클래스 그리고 인터페이스

<1강_C++ 기초>

1. Inline 함수

```
int min(int x, int y)
{
    return x > y ? y : x;
}
```

```
int main()
{
    std::cout << min(5, 6) << '\n';
    std::cout << min(3, 2) << '\n';
    return 0;
}
```

```
inline int min(int x, int y)
{
    return x > y ? y : x;
}
```

```
int main()
{
    std::cout << (5 > 6 ? 6 : 5) << '\n';
    std::cout << (3 > 2 ? 2 : 3) << '\n';
    return 0;
}
```

- Inline 함수를 사용하면 함수의 Body 부분을 함수 호출 부분에 복사되어 실행됩니다.

- 그러므로 함수 호출을 하는 과정에서 생기는 오버헤드가 사라져 속도가 빨라집니다.

- 현대의 컴파일러는 자동으로 Inline을 하기 때문에 권장 사항 일 뿐입니다.

2. new / delete

- 윤성우 선생님께서 앞으로 C++에서 malloc과 free를 호출하는 일은 없어야 한다고 말씀하셨습니다.
- 학교에서 c언어를 사용해서 malloc / free가 익숙할 뿐 , new / delete가 훨씬 간편한 문법입니다.

1) malloc / free의 단점을 해결한 new / delete

- malloc으로 선언할 때 malloc의 return형이 void 형이므로 적절한 강제 형 변환을 반드시 해야 합니다.
- 무조건 바이트 크기 단위로 할당해야 합니다.

2) 기본 자료형 new

```
//int* arr = (int*)malloc(sizeof(int) * 10);
int* arr = new int[10];
for(int i=0 ; i<10; i++)
{
    arr[i] = i+10;
    cout << arr[i] << " ";
}

$ ./a.exe
10 11 12 13 14 15 16 17 18 19
```

- 강제 형 변환을 하지 않습니다.
- 알아서 해당 자료형 만큼 배열의 크기를 힙에 할당하므로 sizeof연산을 하지 않아도 됩니다.

3) 다양한 자료형 new

```
pair<int,int>* arr2 = new pair<int,int>[10];
for(int i=0; i<10; i++)
{
    arr2[i].first = i+10;
    arr2[i].second = i+20;
    cout << arr2[i].first << "/" << arr2[i].second << " ";
}
```

```
$ ./a.exe
```

```
10/20 11/21 12/22 13/23 14/24 15/25 16/26 17/27 18/28 19/29
```

- int를 선언할 때와 완벽하게 동일합니다.
- 자동으로 pair<int,int>의 크기만큼 10개의 heap 영역을 할당합니다.

4) 배열의 동적 할당

*** 내가 데이터라고 생각한 개념을 윤성우 선생님은 객체라고 표기하셨습니다. ***

① 그림1

```
short* arr = new short[5];
int* arr2 = new int[5];
double* arr3 = new double[5];
printf("%d %d %d" , sizeof(arr) , sizeof(arr2) , sizeof(arr3));
```

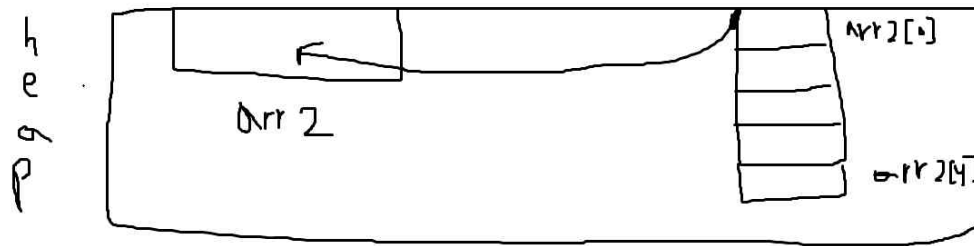
```
$ ./a.exe
```

```
8 8 8
```

```
printf("%p %p %p" , &arr2 , arr2 , &arr2[0]);
```

```
0xffffcbf8 0x60005ceb0 0x60005ceb0
```

- 가장 익숙한 int형을 이용하여 동적 할당 구문을 완벽하게 분석해 보겠습니다.
- 그림을 참조하면 , 다양한 자료형으로 힙에 동적으로 선언한 배열도 가리키는 포인터 변수의 크기는 8 bytes로 동일합니다.
- arr2는 힙에 저장된 포인터 변수이며 int* arr2로 인해 해당 포인터 변수가 메모리상에서 가리키는 데이터는 힙에 저장되어 있으며 int형 데이터입니다.
- int형으로 5개의 원소를 배열형태로 힙에 연속적으로 할당하고 , 8 bytes 크기의 포인터 변수(arr2)에 해당 배열의 가상 주소를 저장합니다.
- 포인터 변수 arr2에 저장된 값(arr2)과 포인터 변수 arr2이 참조 하는 배열의 가상 주소(&arr2[0])는 같은 값을 갖지만 , 포인터 변수 arr2의 가상 주소(&arr2)는 전혀 다른 값을 가집니다.



- 결론적으로 , 위의 동적 할당 선언방식은 heap에 2가지 데이터를 동시에 만드는데 , 먼저 어떤 객체(변수 OR 배열 OR class의 객체)에 대한 메모리 공간을 할당하고 , 해당 객체를 가리키는(참조하는) 언제나 8 bytes의 크기를 갖는 포인터 변수를 할당하고 해당 객체의 주소를 저장하여 연결시킵니다. 또한 이 개념은 일반적인 배열에서만 적용되는 개념이 아니라 struct와 class에서도 동일하게 적용됩니다. 이 내용을 컴퓨터 공학과 2학년 때 배웠지만 이제야 완벽하게 이해합니다.

② 그림2

```
double* arr = new int[];
```

- C++ 컴파일러는 포인터 연산의 가능성 여부를 판단할 때 , 포인터의 자료형을 기준으로 판단하지 , 실제 가리키는 객체의 자료형을 기준으로 판단하지 않습니다.
- 불안정한 형식은 사용할 수 없다고 ERROR가 발생합니다. ERROR가 나는 이유는 double형 객체(데이터)를 가리켜야 하는 포인터에 int형 객체(데이터)가 저장되어 있기 때문입니다.

3. const

- = 어떤 객체가 상수화되면 컴파일러에 따라 메모리의 data영역 OR code영역으로 이동합니다.
- 고수가 되고 , ERROR를 피하기 위해서 확실한 불변은 const로 설정해야 합니다.
- const는 상수이므로 반드시 초기화하며 선언해야 합니다.

1) const 포인터 변수

- 포인터가 저장하는 것은 어떤 변수의 가상 주소

① const int* ptr

- = 포인터에 저장된 가상 주소에 저장된 값을 변경 할 수 없습니다.

```
int a;  
const int* ptr = &a;  
*ptr = 3;  
test.cpp:32:12: error: assignment of read-only location '* ptr'  
                *ptr = 3;  
                ^
```

② int* const ptr

- = 포인터에 저장된 가상 주소를 변경할 수 없습니다.
- = 다시 말해 해당 포인터가 가상 메모리의 다른 가상 주소를 가리킬 수 없습니다.

```
int a,b;  
int* const ptr = &a;  
ptr = &b;  
test.cpp:32:12: error: assignment of read-only variable 'ptr'  
                ptr = &b;  
                ^
```

2) const method

```
class Str
{
public :
    int n;
    void Change() const
    {
        n=10;
    }
};

int main()
{
    Str obj;
    obj.Change();
    cout << obj.n;
}
```

- Change() method 뒤에 const를 붙임으로써 해당 method를 const method로 변경시켰습니다.
- **const method는 객체의 멤버 변수에 저장된 값을 변경하지 못합니다.** 변경하면 아래의 컴파일 에러가 발생합니다.
- 그러므로 **const method 내에서는 일반 method는 올 수 없고 const method만 올 수 있습니다.** const method가 아닌 일반 method는 멤버 변수를 변경 시킬 우려가 있기 때문에 사전에 컴파일러가 차단시킵니다.

```
test.cpp:34:15: error: assignment of member 'Str::n' in read-only object
    n=10;
    ^~
```


3) const 객체

= 객체를 만들 때 const를 붙여 해당 객체의 모든 내용을 변경하지 않겠다고 선언합니다.

- const의 성질로 인해 해당 객체에서는 const method만 호출이 가능하고 , 일반 method는 호출이 불가능 합니다.

```
const str obj;  
const str obj(20,30);
```

4. static

1) static을 이해하기 위해 포인터를 다시 정리해봅니다.

```
int stack_arr[5];
int* heap_arr = new int[5];
for(int i=0; i<5; i++)
{
    stack_arr[i] = i+10;
    heap_arr[i] = i+20;
}

//1. stack
printf("*** stack ***\n");
printf("배열 전체 : %p ", stack_arr);
printf(", 원소의 주소 : ");
for(int i=0; i<5; i++)
{
    printf("%p ", &stack_arr[i]);
}

//2. Heap
printf("\n*** Heap ***\n");
printf("배열 전체 : %p ", heap_arr);
printf(", 원소의 주소 : ");
for(int i=0; i<5; i++)
{
    printf("%p ", &heap_arr[i]);
}
```

```
$ ./a.exe
*** stack ***
배열 전체 : 0xffffcbf0 , 원소의 주소 : 0xffffcbf0 0xffffcbf4 0xffffcbf8 0xffffcbfc 0xffffcc00
*** Heap ***
배열 전체 : 0x600000450 , 원소의 주소 : 0x600000450 0x600000454 0x600000458 0x60000045c 0x600000460
```

- 정적 할당과 동적 할당의 차이는 결국 Stack에 할당하느냐 , Heap에 할당하느냐에 대한 차이 입니다. 그러므로 가상 주소를 확인하는 방식도 당연히 동일합니다.

- 그림을 확인하면 , 두 배열의 주소를 확인하는 방식이 완벽하게 동일함을 알 수 있습니다.

2) static 멤버 변수

```
class str
{
public:
    int a;
    double b;
    static int c;
};
int str::c = 3;

str obj[3];
printf("1. %p %p %p\n" , &obj[0] , &obj[1] , &obj[2]);
printf("2. %p %p %p\n" , &obj[0].c , &obj[1].c , &obj[2].c);

str* ptr = &obj[0];
printf("3. %p\n" , ptr);

str* arr = new str[3];
printf("4. %p %p %p\n" , &arr[0] , &arr[1] , &arr[2]);
```

```
$ ./a.exe
1. 0xfffffcbef0 0xfffffcbf0 0xffffcc00
2. 0x100402010 0x100402010 0x100402010
3. 0xfffffcbef0
4. 0x60005ce80 0x60005ce90 0x60005cea0
```

① 1번 출력

- 메모리의 Stack 영역에 클래스에 대한 객체 3개를 선언하고 , 각각의 객체의 가상 주소를 출력합니다.
- 출력을 확인하면 be0 -> bf0 -> c00이므로 각각의 객체는 16Bytes의 크기를 할당받는 것을 알 수 있습니다. 이는 int가 double로 인해 8bytes가 되어 8*2 bytes의 크기를 할당 받게 됩니다.
- static은 객체의 메모리 영역에 포함되지 않음을 알 수 있습니다.

② 2번 출력 (static 멤버 변수)

- 객체 3개에 대해 static 멤버 변수 c의 가상 주소를 확인해 보면 모두 같은 가상 주소를 갖고 , 이는 메모리의 Data 영역에 저장 됩니다.
- 다시 말해 , static 멤버 변수는 각각의 객체의 메모리 영역에 포함되지 않고 따로 메모리의 Data 영역에 저장되어 프로그램이 종료될 때 까지 해당 클래스에서 만든 모든 객체가 공유합니다. 그러므로 0번 객체에서 c의 값을 1증가시키면 다른 객체에서도 c의 값은 1증가 됩니다. 그로 인해 static 멤버 변수를 클래스 변수라고 부릅니다.

③ 3번 출력

- Stack 영역에 할당한 객체 0번의 주소를 저장하는 포인터

④ 4번 출력

- Stack 영역에 할당한 obj[]객체와 무관하게 Heap 영역에 동적으로 arr[]객체를 할당합니다. 그러므로 1번 출력과 비교해보면 가상 주소의 차이를 확연하게 알 수 있습니다.

3) static 멤버 변수 확인

```
class str
{
    public :
        int a;
        double b;
        static int c;
};
int str::c = 3;

printf("%d" , str::c);
```

```
$ ./a.exe
3
```

- static 멤버 변수는 클래스 밖에서 초기화를 해줍니다.
- 아래의 printf문처럼 적어주면 클래스 멤버 변수의 값을 확인할 수 있습니다.
- str::c는 클래스 멤버 변수이고 , 결국에는 변수이므로 값을 변경할 수 있습니다.

4) static 멤버 함수

- static 멤버 함수 내에서는 static 멤버 변수와 static 멤버 함수만 호출이 가능합니다.

5. 멤버 참조

1) Stack

- 스택에 정적으로 객체를 할당하면 멤버 변수나 method에 접근할 때 obj.member , obj.Func()로 접근합니다.

2) Heap

- 힙에 동적으로 객체를 할당하면 멤버 변수나 method에 접근할 때 obj->member , obj->Func()로 접근합니다.

6. 헤더파일 , 소스파일 , namespace

1) header file

- 클래스의 멤버 변수 , 클래스의 method 이름을 저장합니다.

2) source file

- 연관된 헤더 파일의 클래스의 method body를 구현합니다.
- source file에 필요한 header file은 include로 반드시 적어주어야 합니다.
- main 함수는 연관된 헤더 파일 없이 독립적인 source file로 구현합니다.

3) namespace

- 기업에서는 반드시 사용하는 듯?

<2강_클래스의 기본>

1. 생성자 (Constructor) : Unity의 Awake()

= 객체를 생성하는 동시에 객체의 멤버 변수들을 초기화 시킬 수 있습니다.

- 객체를 생성할 때 한 번만 호출됩니다. 객체를 생성 할 때 안전한 값으로 초기화를 시켜줍니다.

- 객체를 만들고 생성자의 호출로 멤버변수의 초기화까지 완료되어야 진정한 객체라고 표현할 수 있습니다.

- 생성자는 존재하는 객체에 존재하는 모든 것을 초기화해야 합니다. (effective c++ p71)

- 생성자 선언(내가 만든 용어) : 초기화를 하는 생성자를 만들면서 객체를 만드는 선언. (ex : str obj(20,30))

- Default 선언(내가 만든 용어) : 생성자를 만들지 않고 객체를 만드는 선언. (ex : str obj)

- 지금까지 객체를 선언할 때(구조체) Default 생성자만을 사용했었습니다. 하지만 생성자를 만들고 생성자에서 멤버의 초기화가 1개 이상 일어난다면 Default 선언을 할 수 없습니다.

- 사실 Default 선언도 비어있는 생성자를 프로그램이 자동으로 호출합니다.

- Method의 형태지만 Return 형이 없습니다.

- Method의 형태이므로 오버로딩이 가능 합니다.

2. 소멸자 (Destructor) : Unity의 Destroy()

= 객체를 소멸시킬 때 반드시 호출되는 return 형이 없는 method

- 소멸자를 이해하기 위해서는 객체의 소멸시점을 이해해야 합니다. 다행히도 객체 또한 우리가 사용해 왔던 변수 , 구조체 변수의 소멸 시점과 완벽하게 동일합니다. 전역 변수는 프로그램 종료 시점까지 생존해 있으므로 따로 메모리를 해제 하지 않아도 괜찮습니다.

- 생성자는 객체를 생성할 때 관련된 인자를 포함한 코드를 적어주어야 하지만 소멸자는 객체가 소멸되는 시점에 자동으로 호출되기 때문에 클래스 내에서 소멸자를 구현해주면 프로그램에서는 객체가 소멸될 때 자동으로 구현한 소멸자를 호출해줍니다.

- 생성자에서 동적 할당을 한 변수를 소멸자에서 해제시키도록 개발자가 구현해야 합니다.

ex) 'int* arr = new int[1000];'로 생성자에서 선언을 했다면 소멸자에서 'delete []arr'로 메모리를 해제시킵니다.

- 클래스 명이 AAA라면 , 소멸자는 ~AAA()의 형태입니다.

- 생성자와 마찬가지로 소멸자를 만들지 않는다면 , 자동으로 Default 소멸자가 생성됩니다.

3. this 포인터

```
class str
{
public :
    int a;
    void Func()
    {
        this->a = 5;
    }
};

int main()
{
    ios_base :: sync_with_stdio(false);
    cin.tie(NULL);

    str obj;
    obj.Func();
    cout << obj.a;
    return 0;
}
```

- 클래스에서 this 포인터는 해당 클래스를 틀로 만든 객체를 의미합니다.
- 클래스는 틀이고 객체를 만들면 클래스를 이용합니다.
- 만들어진 객체가 클래스에서 자신을 의미하는 포인터로 this를 사용합니다.
- this는 결국 클래스의 멤버를 가리키고 지역변수는 가리키지 못하기 때문에 클래스의 method에서 만든 지역변수와 클래스의 멤버 변수가 이름이 같아도 this를 이용하면 차이를 나타낼 수 있습니다. (윤성우 c++ 198 참조)

5. 정보은닉과 캡슐화

1) 정보은닉

정보은닉(information hiding)이란 다른 객체에게 자신의 정보를 숨기고 자신의 연산만을 통해 접근을 허용하는 것으로, 클래스 외부에서 특정 정보에 접근을 막는다는 의미이다.^[1]
은닉성(隱匿性)이라고도 한다.

정보은닉(Information hiding)은 모든 객체지향 언어적 요소를 활용하여 객체에 대한 구체적인 정보를 노출시키지 않도록 하는 기법으로 객체지향프로그래밍에서 중요한 개념이다.
^[2] 정보은닉의 개념은 캡슐화와 유사한 부분이 많아 같은 개념이라고 많이들 오해하지만 시스템 처리 원리를 기준으로 본다면 다른 개념이다. 하지만 유사하다고 생각해도 큰 문제는 없으나 처리 원리에 대해 각각의 차이점을 알아두는 것이 좋다.

- **제한된 방법으로의 접근만 허용을 해서 잘못된 값이 저장되지 않도록 도와야** 하고 , 또 실수를 했을 때 , 실수가 쉽게 발견되도록 해야 하기 때문에 정보은닉을 사용합니다.

2) 캡슐화

캡슐화(**영어**: encapsulation)는 **객체 지향 프로그래밍**에서 다음 2가지 측면이 있다.^{[1][2]}

- 객체의 속성(data fields)과 행위(메서드, methods)를 하나로 묶고,^{[3][4]}
- 실제 구현 내용 일부를 외부에 감추어 은닉한다.^{[5][6]}

= **관련 있는 함수와 변수를 하나의 클래스 안에 묶는 개념**입니다.

- ‘관련 있는’이라는 용어의 범위를 결정하는 일은 쉽지 않기 때문에 캡슐화 과정은 어렵습니다.
- 캡슐화 과정에는 정보은닉이 기본적으로 포함됩니다.

<3강_상속>

1. 상속

- 상속을 함으로써 불필요한 코드의 재사용을 줄일 수 있고 , 변경을 최소화 할 수 있습니다.
- 부모(기초) 클래스를 상속받으면 자식(유도) 클래스는 부모 클래스의 모든 **멤버(멤버 변수 & method)**를 물려받습니다.

2. protected

= **protected로 선언된 멤버는 자식 클래스에서 접근이 가능합니다.**

- public과 private에 비해 거의 사용되지 않습니다.
- 접근 지정자를 적진 않은 default 접근 지정자를 protected로 알고 있었는데 private입니다.

3. 세 가지 형태의 상속

1) public 상속

- 윤성우 선생님은 C++의 상속은 public 상속만 있다고 생각해도 무방하다고 하십니다.
- **private를 제외한 나머지 접근 제어자인 protected와 public 멤버를 자식 클래스에서 그대로 상속받습니다.**

2) protected 상속

- protected 보다 범위가 넓은 멤버를 protected로 변경 시켜 상속받습니다.
- private는 상속 받지 못하고 , protected와 public을 protected로 변경 시켜서 상속받습니다.

3) private 상속

- private 보다 범위가 넓은 멤버를 private로 변경 시켜 상속받습니다.
- private는 상속 받지 못하고 , protected와 public을 private로 변경 시켜서 상속받습니다.

4. 자식 클래스의 생성자

- 자식 클래스의 생성자에서는 자신의 멤버뿐만 아니라 반드시 부모 클래스의 멤버 까지 모두 초기화를 해야 합니다. 이 때 부모 클래스의 생성자를 호출해서 부모 클래스의 멤버를 초기화해야 합니다. 부모 클래스의 멤버는 이니셜라이저로 초기화 합니다. (이렇게 초기화를 해지 않으면 컴파일 에러 발생) (윤성우 c++ 291)

1) 상속 예제 및 멤버 변수 출력

```
class parent
{
public :
    int a;
    int b;
    parent(int value_1 , int value_2)
    {
        a = value_1;
        b = value_2;
    }
};

class child : public parent
{
public :
    int c;
    child(int value_1 , int value_2 , int value_3) : parent(value_1,value_2)
    {
        c = value_3;
    }
};
```

```
child baby(11,22,33);
cout << baby.a << " " << baby.b << " " << baby.c;
```

```
$ ./a.exe
11 22 33
```

2) 부모 클래스의 객체와 자식 클래스의 객체에 대한 가상 주소

```
parent mom(5,6);
child baby(11,22,33);
printf("%p %p\n" , &mom.a , &mom.b);
printf("%p %p %p" , &baby.a , &baby.b , &baby.c);
```

```
0xfffffcc18 0xfffffcc1c
0xfffffcc0c 0xffffcc10 0xffffcc14
```

- 상속을 받아도 부모클래스의 객체와 자식클래스의 객체의 멤버에 대한 가상 주소는 독립적입니다.

5. 자식 클래스의 객체 생성 및 소멸 과정

*** 기본적으로 생성자와 소멸자는 오버로딩이 가능하므로 , 각각의 매개변수에 맞는 생성자와 소멸자가 호출됩니다. ***

1) 객체 생성 과정

- 자식 클래스가 생성되면 자신의 생성자를 호출하고 , 그 후에 100% 부모의 생성자를 호출합니다. (순서도 중요합니다.)
- 자식 클래스의 생성자에서 부모 클래스의 생성자(이니셜라이저)를 명시하지 않으면 부모 클래스의 오버라이딩 된 생성자 중에서 void형 생성자를 호출합니다.

2) 객체 소멸 과정

- 자식 클래스가 소멸되면 100% 부모의 소멸자를 호출하고 , 그 후에 자식의 소멸자를 호출합니다. (순서도 중요합니다.)
- 객체는 스택 구조에 존재하기 때문에 늦게 생성된 부모 객체가 먼저 소멸됩니다.

6. 부모 클래스와 자식 클래스 대신에 Base Class 와 Derived Class로 사용해야 하는 이유 (그러나 부모와 자식이 편함)

- 자료형을 타입으로 표현하겠습니다.
- 상속은 언제나 is-a 관계여야 하며 , 완벽한 포함관계를 의미합니다. 예를 들어 Student is a Person을 생각하면 모든 학생은 사람의 일종입니다. 단 하나의 예외조차 없습니다. 그러므로 작은 원에 Student를 넣고 , 작은 원을 포함하는 큰 원에 Person을 넣어 포함관계를 형성할 수 있습니다.
- Person은 Student보다 더 일반적인 개념이고 Student는 보다 특수한 개념입니다.
- 상속에서 큰 원에 해당하는 클래스는 작은 원에 해당하는 클래스를 모두 가리킬 수 있습니다.
- 상속에서의 클래스 간에 관계는 포함관계이므로 부모-자식 관계로 생각하면 혼란이 생길 수 있고 , 상속관계에서의 참조 관계를 생각해 보아도 Base Class 와 Derived Class로 생각하는 것이 올바릅니다.

<4강_다형성>

1. 다형성의 정의 및 기본 성질

1) 정의

- = 부모 클래스에서 정의된 method가 자식 클래스에서 override 되어 대체하여 사용 되는 기능입니다.
- = 클래스(객체)가 다양한 형태로 동작할 수 있는 상태를 다형성이라 합니다.

2) 성질

- = 부모 클래스의 형으로 자식 클래스의 객체를 만들 수 있는 것이 다형성의 기본입니다.

```
class Parent
{
public:
    int a;
    int b;
    void Func()
    {
        cout << "parent\n";
    }
};

class Child : public Parent
{
public:
    int c;
    void Func()
    {
        cout << "child\n";
    }
};
```

```
Parent* obj = new Child();
obj->Func(); //parent를 출력합니다.
```

2. 다형성의 성질을 적용 하지 않음 vs 다형성의 성질을 적용함 (실무 경험이 없어서 확실한 답변을 내놓지 못했고 , 책과 모호한 인터넷 글 작성)

```
class Parent
{
public:
    int a;
    int b;
    void Parent_Func()
    {
        cout << "Parent_Func called";
    }
    void Func()
    {
        cout << "parent\n";
    }
};

class Child : public Parent
{
public:
    int c;
    void Child_Func()
    {
        cout << "Child_Func called";
    }
    void Func()
    {
        cout << "child\n";
    }
};
```

```
//1. 다형성 X
Child* same = new Child();
same->Func(); //Child를 출력합니다.
same->Parent_Func(); //Parent_Func called를 출력합니다.
same->Child_Func(); //Child_Func called를 출력합니다.

//2. 다형성 O
Parent* diff = new Child();
diff->Func(); //parent를 출력합니다.
diff->Parent_Func(); //Parent_Func called를 출력합니다.
//diff->Child_Func(); ERROR
```

1) 다형성 X

- Child는 Parent의 모든 멤버를 상속받기 때문에 3가지 함수를 모두 호출할 수 있습니다.
- Child 형이므로 , Func()의 경우 오버라이딩 된 함수를 호출합니다.

2) 다형성 O

- C++ 컴파일러는 포인터 연산의 가능성 여부를 판단 할 때, 포인터의 자료형을 기준으로 판단하지 , 실제 가리키는 객체의 자료형을 기준으로 판단하지 않습니다.
- 다형성을 적용하여 Parent 형으로 객체를 생성하면 , Parent 클래스에 존재하는 멤버에만 접근이 가능하며 , Child 클래스에 존재하는 멤버에 접근하면 컴파일 에러가 발생합니다. 정리하면 , 포인터 형에 해당하는 클래스에 정의된 멤버에만 접근이 가능합니다.
- Func()를 호출하는 경우 , 부모클래스의 Func()를 호출합니다. 여기서 의문이 발생합니다. 아래에 의문에 대해서 고찰해보겠습니다.

3. 다형성에서 발생한 의문

1) 부모의 타입으로 자식 객체를 형성하는 경우에 힘들게 만든 자식 클래스의 멤버에 접근할 수 없는데 왜 굳이 사용할까?

2) 부모의 타입으로 자식 객체를 형성하고 , 자식의 멤버에 접근할 수 있게 해주는 함수 : 가상함수

<5강_가상함수>

1. 가상함수 (Virtual Function)

```
Parent* diff = new Child();
diff->Func(); //Parent를 출력합니다.
diff->Parent_Func(); //Parent_Func called를 출력합니다.
//diff->Child_Func(); ERROR
```

```
class Parent
{
public:
    int a;
    int b;
    void Parent_Func()
    {
        cout << "Parent_Func called\n";
    }
    virtual void Func()
    {
        cout << "parent\n";
    }
};
```

```
Parent* diff = new Child();
diff->Func(); //Child를 출력합니다.
diff->Parent_Func(); //Parent_Func called를 출력합니다.
//diff->Child_Func(); ERROR
```

- 객체 diff는 부모인 Parent의 타입으로 만들고, 객체를 자식인 Child의 타입으로 가리킵니다. 그로 인해 diff는 Parent 클래스의 모든 멤버에는 접근이 가능하지만, Child 클래스의 멤버에는 접근이 불가능합니다.

<가상 함수 사용 이유>

- Child 클래스에 접근하지 못하므로 Func()를 호출하면 부모 클래스의 Func()가 호출되어 Child 클래스의 Func()는 호출되지 않습니다. 힘들게 Child 클래스를 구현했지만 자식에 접근하지 못하여 상속 + 다형성의 이유가 퇴색됩니다.

<가상 함수의 기능>

- Child 클래스에서 오버라이딩을 통해 구현한 Func() 함수를 호출하기 위해서는 Parent 클래스의 Func()에 virtual을 붙여 가상함수로 만들어줍니다. 그러면 Parent 클래스의 Func()를 가상화 시켜버려 무시하고, Child 클래스의 Func()가 호출됩니다.

- 정리하면, 가상 함수로 선언한 함수는 포인터의 자료형을 기반으로 호출대상을 결정하지 않고, 포인터 변수가 실제로 가리키는 객체를 참조하여 호출의 대상을 결정하게 됩니다.

- 여전히 Child 클래스에만 존재하는 Child_Func()는 Parent 클래스의 함수를 오버라이딩 한 함수가 아니므로 호출시 컴파일 에러가 발생합니다.

2. 가상함수의 성질

- 가상함수를 overriding 하는 모든 method는 가상함수가 됩니다. 그러므로 , 굳이 virtual을 붙이지 않아도 자식 클래스의 해당 method는 가상함수가 되지만 가독성을 위해 virtual을 선언하여 가상함수임을 알리는 것이 좋습니다.

3. 가상함수의 원리

- 윤성우 Chapter 9-1 나중에 필요하면 읽자

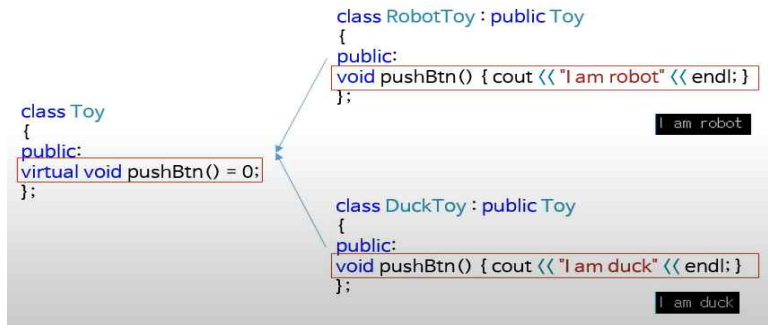
<6강_순수가상함수와 추상클래스 그리고 인터페이스>

1. 순수 가상함수 (Pure Virtual Function)

```
class Toy
{
public:
    virtual void pushBtn() = 0;
};

class RobotToy : public Toy
{
public:
    void pushBtn() { cout << "I am robot" << endl; }
};

class DuckToy : public Toy
{
public:
    void pushBtn() { cout << "I am duck" << endl; }
};
```



- 부모 클래스인 Toy 클래스의 pushBtn()은 순수 가상함수 입니다.
- 그로 인해 , Toy 클래스는 추상클래스가 됩니다.

= 함수의 몸체가 정의되지 않은 가상함수

- 함수에 0을 대입하여 컴파일러에게 해당 함수가 순수 가상함수임을 알립니다. 그로 인해 컴파일러는 함수의 몸체가 정의되지 않았음에도 불구하고 컴파일 오류를 일으키지 않습니다.

Are abstract methods and pure virtual functions the same thing?

Yes, they are the same thing. In C++, an abstract method is just another way of describing the characteristics of a pure virtual function. Both just mean a method with no implementation provided that needs to be implemented in a sub-class before the class can actually be instantiated.

The situation with pure virtual functions and abstract classes in C++ is similar as they essentially mean exactly the same thing. Any abstract class must have at least 1 pure virtual function or else it could be instantiated and wouldn't be abstract. Likewise, any class with at least 1 pure virtual function must be abstract because it needs to be extended so that method can actually be implemented.

- 순수가상함수는 추상 메소드(=Abstract Method)와 완전히 동일한 용어 입니다.

2. 추상클래스 (Abstract Class)

- = 하나 이상의 멤버함수를 순수 가상함수로 선언한 클래스를 가리켜 추상 클래스라 합니다.
- 멤버 변수를 만들지 않고 , 모든 멤버 함수가 순수 가상 함수인 추상클래스가 바람직합니다. (인터페이스로 만들기)
- 인터페이스와 비슷한 개념입니다.

3. 인터페이스 (Interface)

```
public interface 청소담당 {  
    public void 청소 ();  
    public void 교실_이동 (ClassRoom 교실);  
}
```

```
public class 창문닦이_v2 implements 청소담당 {  
  
    private ClassRoom 담당_교실;  
    private int 행주_깨끗함 = 100;  
    private double 원덱스_리터 = 10.0;  
  
    public 창문닦이_v2 (ClassRoom 교실) {  
        담당_교실 = 교실;  
    }  
  
    public void 창문_닦기() {  
        while (!담당_교실.창문_깨끗함 ()) {  
            if (행주_깨끗함 <= 10)  
                빨아오기(행주_깨끗함);  
            if (원덱스_리터 <= 1)  
                리필해오기(원덱스_리터);  
            담당_교실.창문_닦기(행주_깨끗함, 원덱스_리터);  
        }  
    }  
  
    @Override  
    public void 청소() {  
        창문_닦기();  
    }  
  
    @Override  
    public void 교실_이동(ClassRoom 교실) {  
        담당_교실 = 교실;  
    }  
}
```

= 모든 메소드가 추상 메소드면 인터페이스입니다. (=모든 함수가 순수 가상함수면 인터페이스)

- 모든 함수가 순수 가상함수이므로 상속 받고 , 오버라이딩해서 기능(Body)을 추가해야합니다.

- 상속 받았는데 기능(Body)을 추가하지 않으면 컴파일 에러?

4. 추상클래스 vs 인터페이스

추상 클래스(Abstract Class)

클래스는 일반 클래스(Concrete Class)와 추상 클래스로 나뉘는데 추상 클래스는 **클래스 구현부 내부에 추상 메서드가 하나 이상 포함되거나 abstract로 정의된 경우**를 말한다. 이러한 정의는 피상적인 것 같다.

공통의 자료형이라고 할 수 있는 추상클래스가 갖고 있는 공통 서비스 함수이다. 다만, 공통 서비스인데 구현이 공통인 것이 아니라 그 서비스라고 하는 목록이 공통인 것을 말한다. 이렇게 정의를 하는게 정확하게 전달하는 의미라고 생각한다.

- 추상 클래스는 new 연산자를 사용하여 객체를 생성할 수 없다.
- 추상 클래스(부모)와 일반 클래스(자식)는 상속의 관계에 놓여있다.
- 추상 클래스는 새로운 일반 클래스를 위한 부모 클래스의 용도로만 사용된다.
- 일반 클래스들의 필드와 메소드를 통일하여 일반 클래스 작성 시 시간을 절약할 수 있다.
- 추상 클래스는 단일 상속만 가능하며 일반 변수를 가질 수 있다.
- 추상 클래스는 **동일한 부모를 가지는 클래스를 묶는 개념으로 상속을 받아서 기능을 확장시키는 것이 목적**이다.

인터페이스(Interface)

인터페이스는 **모든 메서드가 추상 메서드인 경우**를 말한다.

- 인터페이스는 추상 클래스보다 한 단계 더 추상화된 클래스라고 볼 수 있다.
 - 인터페이스에 적는 모든 메서드들은 추상 메서드로 간주되기 때문에 abstract를 적지 않는다.
 - 기존에는 인터페이스에 일반 메서드를 구현할 수 없었지만, 자바8 버전부터 **default 예약어**를 통해 일반 메서드 구현이 가능하다.
- 인터페이스는 static final 필드만 가질 수 있다. 필드를 선언할 때는 **public static final**이 생략되어있다고 생각하자.
 - public static final을 사용하는 이유는 다음과 같다.
 - 구현 객체의 같은 동작을 보장하기 위한 목적
 - 인터페이스의 변수는 스스로 초기화 될 권한이 없음
 - **아무 인스턴스도 존재하지 않는 시점이기 때문**
 - 인터페이스도 추상 클래스와 마찬가지로 new 연산자를 사용하여 객체를 생성할 수 없다.
 - 인터페이스는 **구현 객체가 같은 동작을 한다는 것을 보장하는 것이 목적**이다.
 - 인터페이스는 추상 클래스와 반대로 **다중 상속이 가능**하다.

5. (참고) Unity 인터페이스

1) 정의

= 설계도

- Script = Class를 확장시켜 Script = (Class OR Interface)로 정의할 수 있습니다.
- Interface를 구현하는 Script의 이름 앞에는 'I'를 붙이기로 약속했습니다.
- method 이름만 구현하고 , 아무 기능을 구현하지 않기 때문에 반드시 다른 Script(=Class)에 상속 받아서 사용해야 합니다.
- 어떤 Script(=Class)가 Interface를 상속 받았다면 해당 Script는 Interface에 존재하는 모든 Method를 구현해야 합니다. 이 과정이 Override이지만 C#에서는 Override를 붙이지 않아도 됩니다.

```
public class GoldItem : MonoBehaviour, IItem {
```

- GoldItem(=Class)은 MonoBehaviour(=Class)와 IItem(=Interface)을 상속받습니다.
- GoldItem은 IItem에 존재하는 모든 Method를 구현해야 합니다.

2) 인터페이스를 사용해야 하는 이유 (게임을 예시로 설명)

- 이 게임에는 아이템이 1,000 종류가 있고 플레이어와 닿으면 해당 아이템이 자동으로 사용됩니다.
- 아이템이 자동으로 사용되는 기능은 Use 라는 Method로 구현합니다.

① 기존의 허접한 방식 : If문 내부에 1,000개의 분기 생성

= Player Script에 OnTriggerEnter()함수를 만들고 , 그 안에 1,000개의 if문을 이용하여 조건에 맞는 아이템을 작동시킬 것 입니다. 1,000개의 아이템마다의 Script에는 당연히 Use 함수를 구현해야 합니다.

② Interface를 사용하는 훌륭한 방식 : If문 내부에 단 1개의 분기 생성

= Interface의 다형성과 Interface를 상속 받은 Class는 반드시 Interface의 Method를 구현해야 하는 방식으로 인해 분기 없이 코드를 작성할 수 있습니다. 1,000개의 아이템마다의 Script에는 당연히 Use 함수를 구현해야 합니다.

<허접한 기존의 방식>

```
void OnTriggerEnter2D(Collider2D other)
{
    GoldItem goldItem = other.GetComponent<GoldItem>();

    if(goldItem != null)
    {
        goldItem.Use();
    }

    HealthKitItem healthKitItem = other.GetComponent<HealthKitItem>();

    if(healthKitItem != null)
    {
        healthKitItem.Use();
    }
}
```

<Interface를 사용한 훌륭한 방식>

```
void OnTriggerEnter2D(Collider2D other)
{
    IItem item = other.GetComponent<IItem>();

    if(item != null)
    {
        item.Use();
    }
}
```

- Reference : <https://www.udemy.com/course/retr0-unity/learn/lecture/8609448#overview>